



Elastoformer: Enabling Dynamic Adaptivity via Elastic Model Transformation

Sudaksh Kalra
University of Amsterdam
Amsterdam, Netherlands
s.kalra@uva.nl

Dolly Sapra
University of Amsterdam
Amsterdam, Netherlands
d.sapra@uva.nl

Abstract

EdgeAI systems are increasingly employing computer vision applications to enable intelligent, on-device decision-making in real-time. However, these deployments face highly dynamic operational conditions, with fluctuating constraints on latency, power availability, and memory resources. Deep Neural Networks (DNN), which follow fixed computational execution flows, lack the flexibility to adapt to such variability, resulting in inefficient and suboptimal performance in edge scenarios. This underscores the need for architectures that are not only efficient but also dynamically scalable at runtime. In this paper, we propose *Elastoformer*: A framework that transforms conventional neural networks (NN) into *Elastic NN* capable of real-time elastic inference. Unlike the conventional bag-of-models approach, which requires maintaining multiple independent models for different operating conditions, *Elastoformer* offers a single, modular solution that dynamically switches between multiple modes of operation at runtime, adapting efficiently to the changing computational budgets of edge devices without the overhead of managing separate models. Experiments reveal that our framework achieves up to 85% reduction in computation FLOPs, 50% reduction in latency and 76% reduction in memory overhead, while showcasing the architecture agnostic nature of the framework across both Vision Transformers and CNNs. Our code is available at <https://github.com/sudaksh14/Elastoformer>.

CCS Concepts

• **Computing methodologies** → **Neural networks**; • **Computer systems organization** → **Embedded software**; • **Software and its engineering** → **Software performance**.

Keywords

EdgeAI, Elastic Inference, Dynamic Neural Network, Vision Transformer, Convolution Neural Network

ACM Reference Format:

Sudaksh Kalra and Dolly Sapra. 2025. *Elastoformer: Enabling Dynamic Adaptivity via Elastic Model Transformation*. In *The Tenth ACM/IEEE Symposium on Edge Computing (SEC '25)*, December 3–6, 2025, Arlington, VA, USA. ACM, New York, NY, USA, 14 pages. <https://doi.org/10.1145/3769102.3770612>

1 Introduction

Edge devices are increasingly being deployed for computer vision applications across diverse domains, including industrial automation, healthcare, autonomous vehicles and smart cities [2, 32]. These devices enable low latency, real-time decision making without relying on cloud connectivity. They also process user data locally, making them well-suited for applications with strict security and privacy requirements. EdgeAI systems, in this context, refer to artificial intelligence solutions that operate directly on such edge devices, enabling on-device inference or learning under constrained computational, memory, and energy resources. Despite multiple benefits, EdgeAI systems face significant challenges [9, 12, 38, 39] due to limited computational resources, energy constraints, and storage limitations. Moreover, the availability of these resources is often dynamic, influenced by competing workloads and changing runtime conditions. To operate reliably under such variability, EdgeAI models must be capable of adapting their computational demands to the current state of the system.

To address these limitations, modern EdgeAI systems must incorporate more efficient neural networks that require reduced computational and memory resources. Furthermore, these networks should be capable of dynamically adapting their computational demands in response to changes in system state. The majority of the computer vision networks are monolithic in design with a fixed computational workflow. Traditional developments for improving computer vision performance typically involve increasing model complexity by adding parameters and raising the number of Floating Point Operations (FLOPs) [4, 14, 33].



This work is licensed under a Creative Commons Attribution 4.0 International License.

SEC '25, Arlington, VA, USA

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2238-7/25/12

<https://doi.org/10.1145/3769102.3770612>

In this work, we propose *Elastoformer*, a dynamic neural network architecture that maintains competitive performance while adapting to continuously evolving system conditions of EdgeAI systems. *Elastoformer* provides a unified framework for transforming existing pre-trained models into elastic architectures capable of runtime adaptation. Fig. 1 illustrates the trade-off between performance and resources (FLOPs) achieved by the proposed *Elastoformer*, in comparison with baseline vision architectures. While our method is motivated by the recent success of vision transformers [4], it is model-agnostic and applies equally to convolutional neural networks (CNNs), as also demonstrated in our experiments.

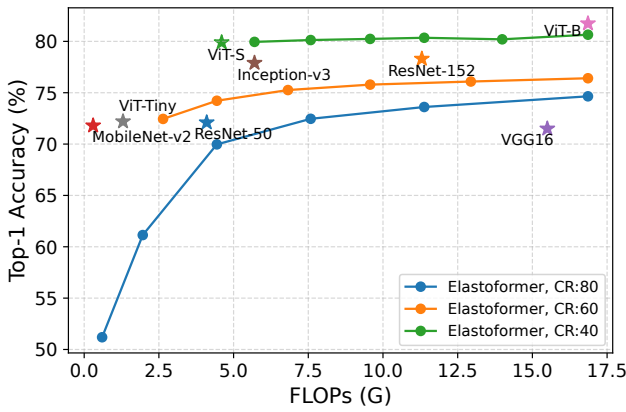


Figure 1: Performance–efficiency Trade-off for Elastoformer compared to baseline vision models on ImageNet

At its core, *Elastoformer* incorporates a two step *Compress* and *Grow* framework to construct an elastic network which internally comprises of multiple *Descendant Networks* (DN). Each DN is capable of serving a different mode of operation for the EdgeAI system. This flexible design enables the instantiation of multiple runtime configurations from a single pre-trained model, supporting a wide range of compression levels to accommodate the dynamic and resource-constrained nature of edge environments.

Our work lies at the intersection of dynamic neural network and neural network compression. A wide range of techniques have been proposed to reduce the computational demands of neural networks, particularly the number of FLOPs, through various compression strategies. These include quantization [10, 17] and pruning [8, 11, 22, 24]. Other approaches include the design of compact architectures using either carefully crafted parameters [15] or automated neural architecture search (NAS) algorithms [5, 31]. While effective at reducing model size and inference cost, these approaches

typically yield multiple independent neural networks or operating modes, each tailored to a fixed set of constraints. As a result, all variants must be stored simultaneously on the device, leading to significant memory overhead. Moreover, since there is no shared representation between modes, switching configurations requires paging entire model weight tensors in and out of memory at runtime [18, 23]. This lack of weight reuse not only increases memory consumption, particularly when the same layer structure appears across multiple modes, but also introduces additional latency during transitions, adversely affecting real-time performance.

This gap in existing approaches underscores the need for dynamic or elastic neural networks [12] that can adapt in real time to the changing compute, memory, and energy constraints inherent to EdgeAI systems. Prior works have explored input-adaptive models that reduce computation based on the nature of incoming data [35, 38, 40]. However, these approaches are generally tightly coupled to specific architectures and primarily exploit the nature of the inputs, offering limited benefits for broader system level variability in EdgeAI applications.

Other efforts, particularly in the context of Vision Transformers (ViTs), employ specialized token merging or manipulation techniques to reduce inference costs [1, 29, 41]. While effective, these methods are static once trained, do not support runtime adaptation to varying resource budgets, and often require substantial retraining to recover baseline accuracy. Consequently, their applicability to dynamic and resource-constrained edge environments is limited.

We address said challenges by enabling elastic transformation of existing networks into multiple operational modes tailored to different compute and memory budgets. *Elastoformer* is specifically designed to support real-time adaptability with minimal memory and latency overhead, making it well-suited for edge computing scenarios where platform conditions may vary dynamically.

The key contributions of this work are as follows:

- We introduce *Elastoformer*: an elastic vision transformer that dynamically adapts its size and computational complexity (FLOPs) based on real-time system constraints.
- We develop a general elastic transformation framework applicable to all neural architecture including CNNs, thus supporting broad deployment scenarios.
- Our approach minimizes memory overhead and transition latency by avoiding redundant storage of multiple independent models.
- We propose a novel weight-sharing growth mechanism that reuses parameters across *Descendant Networks* (DN), enabling efficient multi-mode operation with minimal retraining cost.

These contributions pave the way for more adaptable, resource-efficient neural architectures that align with the practical constraints and variability of real-world edge deployments.

2 Background and Motivation

The rapid proliferation of intelligent edge devices such as smartphones, IoT sensors, robots, autonomous drones and other embedded systems has catalyzed a significant shift in how AI is deployed. Instead of relying solely on centralized cloud infrastructure, there is a growing need to execute AI inference directly on edge devices, a paradigm referred to as Edge AI. This shift is driven by the demand for low-latency responses, reduced bandwidth consumption, enhanced data privacy and improved energy efficiency, all of which are critical in real-time and critical applications such as smart surveillance, autonomous navigation, industrial automation, and wearable health monitoring. We illustrate through a real world use case, how EdgeAI systems often operate in non-stationary environments with fluctuating resource availability and varying input complexity.

2.1 Motivational Example

Consider an autonomous drone equipped with a vision-based perception system for navigation, obstacle avoidance, and safe package delivery. Such a system typically integrates multiple vision tasks including image classification, object detection, and scene understanding, working in tandem with flight-critical subsystems to enable real-time situational awareness and safe decision-making.

Usually the runtime requirements of these systems vary considerably depending on the delivery scenario. Flying through dense urban areas imposes far higher performance demands than navigating open suburban or rural regions, and these demands can further fluctuate depending on temporal factors such as peak delivery hours versus off-peak periods or weekends. For instance, during urban peak hours, the drone must operate in highly dynamic and congested airspace, facing obstacles such as delivery vehicles, other drones, power lines, building edges, and temporary construction cranes. Environmental factors like strong winds, rain, fog, sun glare, or reflective surfaces, further degrade sensor reliability and challenge flight stability. These conditions demand high-accuracy, low-latency perception models capable of real-time obstacle detection, trajectory planning, and collision avoidance. Yet, onboard computational resources must still be shared with other critical subsystems, such as GPS navigation, communication, and payload management, which imposes tight constraints on the vision pipeline.

To illustrate this challenge, Fig. 2 contrasts static and elastic neural networks under fluctuating device power budgets.

In the static case (top), when available power drops (e.g., due to battery discharge), the DNN continues to demand fixed compute power, leading to increased latency or even system malfunction. In contrast, an elastic NN (bottom) scales its requirements in real time to remain within the available budget, enabling seamless perception-assisted flight without compromising safety.

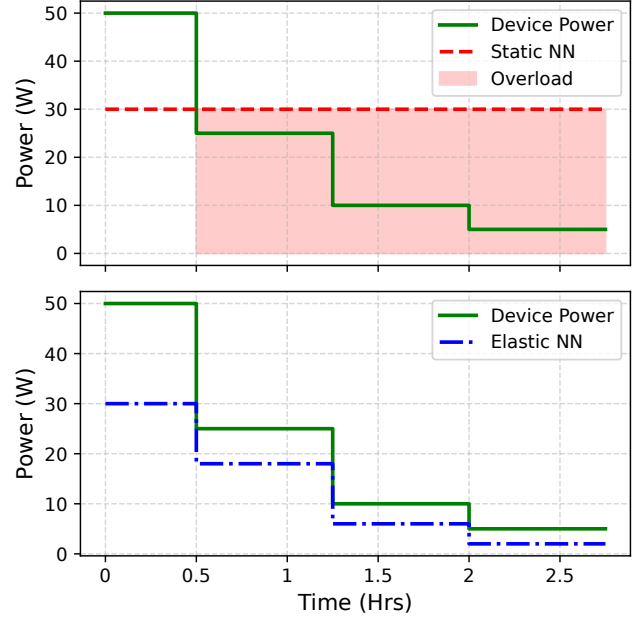


Figure 2: Static (top) vs Elastic (bottom) NN under fluctuating device power budgets

In such dynamic environments, energy and memory resources for vision pipelines are inherently constrained by battery limitations, environmental conditions, temporal traffic fluctuations, and competing onboard processes. This variability underscores the need for computation-aware optimization strategies that adapt vision model execution to both the platform’s resource budget and the complexity of its operating context. Traditional neural networks with fixed architectures are optimized for average or worst-case scenarios, but this rigidity leads to inefficiencies: deploying a high-capacity urban-peak model in open rural regions wastes energy, while relying on a lightweight model during dense urban peak hours risks safety-critical failures.

This scenario is presented purely as a motivational example to illustrate variability in computational and energy requirements for drone deliveries and the potential benefits of adaptive neural networks. It does not represent the exact operating conditions of any specific commercial drone platform.

2.2 Neural Network Elasticity

To operate effectively under dynamic runtime constraints, modern AI systems, particularly those deployed at the edge, must go beyond static optimization and embrace neural network *Elasticity*. Elasticity in this context refers to the ability of a neural network to adapt its computational configuration, such as depth, width, or computational path, in response to runtime factors like latency, energy availability, memory constraints, or input complexity. This capability is especially critical in EdgeAI scenarios, where system conditions fluctuate frequently and over-provisioning for worst-case performance is often inefficient or unsustainable. Elastic models are designed to offer a continuum of operational modes that can be selected dynamically at runtime, enabling the system to tailor its behavior to the current operating context. The overarching goal is to maintain a favorable trade-off between task performance and resource consumption, ensuring efficient and reliable inference even as execution conditions evolve.

A straightforward approach to achieve a flexible run-time behavior involves maintaining a set of independent pre-trained models, each optimized for a different point on the accuracy-efficiency spectrum. At runtime, the system switches between these models depending on the current resource availability or application demand [18, 23]. While conceptually simple, this bag-of-models approach incurs substantial memory overhead, which is especially problematic for edge devices with limited on-board storage. Moreover, runtime switching introduces non-trivial latency due to model loading, initialization, and parameter transfer overheads [7].

In direct contrast, an elastic model does not rely on pre-loading multiple independent networks but rather encapsulates multiple execution paths or scalable configurations within a single shared architecture. This enables the system to make fast, low-overhead adjustments to inference cost while maintaining task accuracy under changing operating conditions. Elasticity can be manifested in various forms. A common strategy is to embed early-exit mechanisms [20, 35] that allow inference to terminate at intermediate layers when sufficient confidence is reached. Another form involves scalable architectures that dynamically adjust width (e.g., number of active channels) or depth (e.g., number of layers processed), reducing the active portion of the network during execution [9, 34, 43, 44]. More advanced designs integrate conditional computation, where only the most relevant parts of the model are activated per input, allowing for finer-grained control over compute and memory usage. While such approaches enable smoother adaptability without full model switching, many are based on custom-designed backbones or handcrafted layer structures, reducing compatibility with modern architectures, particularly

those incorporating Multi-Head Attention (MHA) modules used in ViTs.

A more general solution involves using supernet-based frameworks, where a large over-parameterized network (a supernet) encodes a space of possible sub-networks. During deployment, the system dynamically selects sub-networks suited to the available resources [39]. Despite their flexibility, supernet introduce significant challenges. The memory footprint is often $3\times$ larger than standard models, rendering them impractical for modern architectures like ViTs [4], which are already parameter-heavy.

The challenge in achieving effective elasticity lies in balancing flexibility with maintainability. Highly adaptable models must still ensure consistency in outputs, minimize switching overhead, and avoid significant accuracy degradation when operating in reduced modes. Furthermore, for modern architectures such as Vision Transformers, the design of elastic components must respect the structure of attention layers, which often resist naive pruning or modular scaling. Elasticity must be designed not as an afterthought but as an integral part of the model's architecture, allowing it to degrade gracefully and recover efficiently based on runtime feedback.

In this context, *Elastoformer* is proposed as an elastic neural architecture specifically tailored for deployment on edge devices. Unlike approaches that rely on multiple model variants or heavyweight supernet, *Elastoformer* integrates elasticity directly into the backbone of a single model. These modes are not separate networks but part of a unified model structure, allowing *Elastoformer* to expand or shrink its computational graph with minimal switching overhead. This design enables the system to adapt seamlessly to fluctuating latency and energy constraints while preserving competitive accuracy.

3 Related Works

3.1 Neural Network Compression

Neural Networks have grown rapidly in terms of their computation and energy requirements in recent years. In practice, a larger network is able to generalize better because of more parameters and hence all research was focused on creating deeper and bigger architectures for increased performance [4, 14]. As more efficient deep learning applications are necessary for EdgeAI systems, the need for reducing computations of these networks also arise. Research have focused on methods to produce efficient networks by reducing the number of parameters, which in turn leads to reduced FLOPs. In this regard many research propose an optimized hand-crafted architectures [15, 36, 47] with reduced parameters and provide similar or improved performance than the existing state of the art architectures. However these

approaches involve a custom architectures which are not general to adaptation to other networks and needs a lot of trial and error to estimate the best architecture combination. Quantization is another method which reduces the precision (e.g INT4 or INT8) of model parameters [10, 17] to reduce the network FLOPs. However quantized networks require specialized hardware for actual energy and computational savings and thus are rendered incompatible with most GPUs.

Pruning [10, 11, 13, 21] is another popular approach wherein the redundant parameters in the original network are removed to achieve smaller and more efficient networks without much loss in the performance. NN pruning could be broadly categorized into two types: A) Unstructured and B) Structured Pruning. Firstly, the unstructured approach involves selecting fine-grained independent neurons anywhere in the network, making those parameter weights as zero, thus leading to a sparse networks which are inefficient on most GPUs. Secondly we have structured approach which completely removes entire units or blocks of the network which leads to compact and dense networks with lower parameter count. The structured approach [8, 48] can be implemented on pre-trained models and could generate lighter, faster and efficient networks through little re-training after the pruned connections are removed.

3.2 Dynamic Neural Networks

Several existing studies have proposed dynamic architectures that adapt to evolving runtime conditions in edge environments. Slimmable Neural Networks [43, 44] introduce a simple approach for training a single neural network capable of operating at multiple width configurations, thereby enabling real-time and adaptive trade-offs between accuracy and efficiency. Rather than training separate models for each width setting, a shared network is trained with switchable batch normalization layers to support multiple configurations within a unified architecture. However, this method relies on a custom backbone architecture that depends heavily on batch normalization, which poses a limitation for many state of the art architectures such as Vision Transformers (ViTs) that do not utilize batch normalization layers.

AdaptiveNet [39] enables elastic transformation by initializing a large-scale supernet through Neural Architecture Search (NAS) algorithms. This supernet is deployed on the target edge device, where a profiling algorithm is then employed to extract a sub-network tailored to the device's resource constraints. While this method provides runtime elasticity, the supernet is approximately $3\times$ more overparameterized than the original architecture. This substantial overhead poses a significant bottleneck, particularly for modern architectures such as ViTs, which are already computationally

intensive and challenging to deploy efficiently in edge environments.

LegoDNN [9] introduces a block-grained scaling approach by training multiple interchangeable blocks for the same layers, enabling adaptation to dynamic edge environments. This method is designed to support modular architectures that can be reconfigured at runtime. However, the extent of compression achieved is limited, as the framework focuses solely on the intermediate layers that are not directly connected to the input or output, leaving the remainder of the network uncompressed. Furthermore, LegoDNN is constrained to convolutional neural network (CNN) backbones, as the block-grained design is incompatible with the multi-head attention (MHA) layers in transformer architectures. Consequently, this approach is not applicable to ViTs.

Early-exit networks represent another class of dynamic neural networks that incorporate multiple intermediate output layers throughout the backbone to enable early prediction and reduce overall computation [6, 20, 35]. These methods allow certain inputs to exit early based on confidence estimates, thereby offering dynamic inference paths. However, this introduces additional computational overhead due to the need for confidence evaluation at multiple points. Moreover, the decision to execute or skip modules is typically input-dependent, which limits the robustness of such approaches across the entire data distribution [12]. Certain methods, such as [6], employ application-specific backbones tailored to particular downstream tasks e.g., continuous mobile vision but often fail to generalize across diverse architectures or datasets. Other approaches, such as [40], incorporate more complex exit branches using convolutional or attention mechanisms, further increasing the computational burden associated with dynamic inference and in turn the overall latency.

4 Elastoformer Methodology

We introduce *Elastoformer*, a framework that transforms pre-trained neural networks into elastic architectures capable of real-time adaptation to varying system conditions. *Elastoformer* enables the dynamic scaling of computation through a single unified model that supports multiple runtime configurations, each optimized for different resource constraints such as latency, memory, and power.

As illustrated in Fig. 3, the framework consists of three main stages: **(1) Input Specification**, where the pre-trained model and target edge constraints are defined; **(2) Elastic Transformation**, which constructs a family of sub-networks (termed *Descendant Networks*) using a two-step *Compress* and *Grow* pipeline; and **(3) Deployment and Runtime Selection**, where the most suitable network is selected based on real-time system monitoring. The proposed design allows

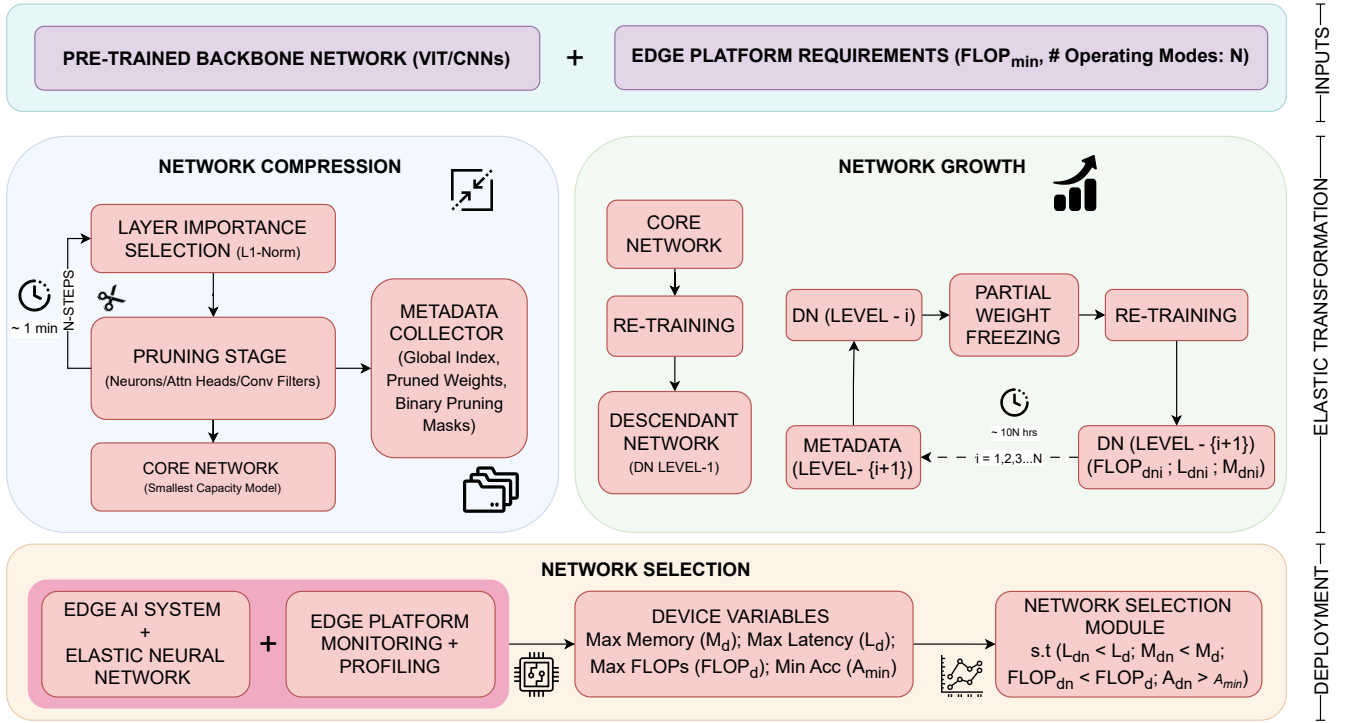


Figure 3: Illustrative Schematic for Elastoformer Framework

Elastoformer to operate under a continuum of performance-efficiency trade-offs while minimizing memory overhead and mode-switching latency. The central component of *Elastoformer* is its two-stage transformation process, *Compress* and *Grow*, which converts a static pre-trained model into an elastic architecture capable of supporting multiple runtime configurations. In the *Compress* stage, we progressively prune the model to generate smaller sub-networks that offer different trade-offs between computational cost and accuracy. In the subsequent *Grow* stage, we incrementally restore pruned weights to produce a hierarchy of larger DN, each benefiting from shared parameters and warm-start initialization. These two stages work in tandem to produce an efficient and scalable set of models, all derived from a single original network. In this section, we describe each stage in detail.

4.1 Neural Network Compression

We apply a structured pruning scheme to compress the original network into multiple progressively smaller variants as shown inside the blue block in Fig. 3. Each pruning step removes a fraction of parameters from the attention and feed-forward layers, while also reducing hidden dimension sizes in the LayerNorm and Patch Embedding layers for greater compression.

Rather than relying on costly second-order gradient methods (e.g., Hessian-based saliency [21, 42]), we adopt a computationally efficient L1-Norm based saliency [22, 37], which generalizes well across model types.

Let N be the number of pruning rounds, $FLOP_B$ be the original FLOPs of the pre-trained network and $FLOP_{min}$ be the minimum available FLOPs for the edge system to run the smallest subnetwork (Core Network) under the specified latency constraints. We can estimate the the desired Compression Ratio, CR and the pruning ratio per round, p as described in (1).

$$\begin{aligned} CR &= FLOP_{min}/FLOP_B \\ p &= 1 - (1 - CR)^{1/N} \end{aligned} \quad (1)$$

The encoder block of the ViT computes the Multi-Head Self-Attention (MHA) as described in (2) following the original implementation [4], where an input image of size $(H \times W \times C)$ is first divided into $N = \frac{HW}{P^2}$ non-overlapping patches of size $(P \times P \times C)$, each linearly projected into embedding dimension d . Here, $Q, K, V \in \mathbb{R}^{n \times d}$ are the query, key, and value matrices derived from the patch embeddings, $W_i^Q, W_i^K, W_i^V \in \mathbb{R}^{d \times d_i}$ are the learned projections for the i -th head and $W^O \in \mathbb{R}^{hd_i \times d}$ is the output projection. Here, $n = N + 1$ is the sequence length (including class token), h is the number of

attention heads and d_i is the embedding dimension for head h_i , where $d_i = d/h$.

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O, \quad (2)$$

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V),$$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_i}}\right)V. \quad (3)$$

In each round, we prune the MHA projections as described in (4) where $M^* \in \{0, 1\}^{d \times d}$ are binary pruning masks computed using an L1-norm-based saliency criterion and $\odot$ denotes element-wise multiplication. Specifically, for each weight matrix (e.g., W^Q , W^K , W^V , or W^O), we compute the average absolute magnitude of each row. Rows with the lowest average L1-norms are assumed to have lower importance and are masked out (set to 0), while the remaining rows are retained (set to 1), enabling structured pruning of the attention projections.

$$\begin{aligned} \tilde{W}^Q &= M^Q \odot W^Q, & \tilde{W}^K &= M^K \odot W^K, \\ \tilde{W}^V &= M^V \odot W^V, & \tilde{W}^O &= M^O \odot W^O \end{aligned} \quad (4)$$

For the Feed-Forward (FF) layers as in (5), we also prune the two-layer MLP as shown in (6):

$$\text{MLP}(x) = W_2 \cdot \phi(W_1 x + b_1) + b_2, \quad (5)$$

where $x \in \mathbb{R}^d$ is the input, $W_1 \in \mathbb{R}^{d_{ff} \times d}$, $W_2 \in \mathbb{R}^{d \times d_{ff}}$, $b_1 \in \mathbb{R}^{d_{ff}}$, and $b_2 \in \mathbb{R}^d$. Here, $\phi(\cdot)$ denotes the non-linear activation (e.g., GELU). We keep $d_{ff} = 4 \times d$ following [4].

$$\tilde{W}_1 = M_1 \odot W_1, \quad \tilde{W}_2 = M_2 \odot W_2, \quad (6)$$

where $M_1 \in \{0, 1\}^{d_{ff} \times d}$ and $M_2 \in \{0, 1\}^{d \times d_{ff}}$ are binary masks, and $\odot$ denotes element-wise multiplication.

As mentioned earlier that our method is architecture agnostic and thus the pruning operation could be extended to CNNs as shown in (7). For a convolutional layer with weight tensor $W_{\text{conv}} \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}} \times K \times K}$, where C_{out} , C_{in} are the number output and input channels respectively and K is the kernel size.

$$\tilde{W}_{\text{conv}} = M_{\text{conv}} \odot W_{\text{conv}}, \quad (7)$$

Here $M_{\text{conv}} \in \{0, 1\}^{C_{\text{out}} \times C_{\text{in}} \times K \times K}$ is a binary mask indicating active kernels.

We store the pruned weights and corresponding masks after each round as pruning metadata. The smallest network obtained finally at the end of this stage is designated as the *Core Network*.

4.2 Neural Network Growth

To recover model capacity and construct higher-performing Descendant Networks (DNs), we incrementally reverse the pruning steps applied during compression as illustrated inside the green block in Fig. 3. Starting from the *Core Network*, the smallest and the most efficient model, we progressively build larger DNs by reintroducing a subset of the previously pruned weights. These weights are restored using the pruning metadata stored during the compression stage, which records their original positions and values.

To avoid redundancy and enable lightweight runtime switching between modes, we do not duplicate any weights. Instead, we employ a dedicated *Weight Sharing* mechanism, as defined in (8). Specifically, W_{L-1} denotes the fixed weights inherited from the previous DN, which are reused directly in the new network and remain frozen. In contrast, W_l contains only the newly introduced parameters from the L -th growth iteration, which are updated during training. This selective update policy allows multiple DNs to coexist with minimal memory overhead while preserving consistency in the shared sub-structures. The composite weight matrix for the Level- L DN, W_L , is given by (8).

$$W_L = W_l + W_{L-1},$$

$$\text{where } W_l(i, j) = \begin{cases} w_{i,j}, & \text{if } (i, j) \in \text{Metadata}_L \\ 0, & \text{otherwise} \end{cases} \quad (8)$$

As illustrated in Fig. 4, each new DN at Level- L is initialized by combining two components: (1) the preserved weights from the previous network that were not pruned, shown as white columns, and (2) the additional weights W_l that were pruned in earlier iterations but are reactivated at iteration L , shown as dark shaded columns. This strategy leverages prior training while providing a warm start for the expanded network, thereby accelerating convergence and improving training efficiency.

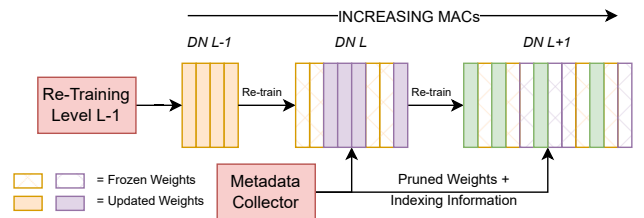


Figure 4: Conceptual Illustration of Partial Weight Freezing for shared-weight elastic inference

To facilitate lightweight network growth without increasing memory overhead and to ensure stable retraining, we adopt two key strategies:

4.2.1 Partial Weight Freezing. To ensure the shared weights W_{L-1} are not modified during training of the Level- L DN, we apply partial weight freezing through gradient masking as in (9). Specifically, we use a binary mask G to zero out the gradients of shared parameters during backpropagation:

$$W_L \leftarrow W_L - \eta \cdot (G \odot \nabla \mathcal{L}) \quad (9)$$

where $G_{ij} = 0$ for all entries belonging to W_{L-1} (shared weights), and $G_{ij} = 1$ otherwise. This guarantees that only the newly added weights are trainable, preserving compatibility with lower modes and avoiding unnecessary memory duplication.

4.2.2 Selective Gradient Clipping. To stabilize training, particularly for large datasets like Imagenet, gradient clipping is utilized to avoid exploding gradients [27]. We apply a selective gradient clipping exclusively to the newly added weights to avoid the exploding gradient only for the parameters which are updated in training. Using the same binary mask G as in (9), we restrict clipping to non-shared parameters. This prevents gradient explosion during re-training while maintaining the integrity of the frozen weights inherited from previous DNs.

4.3 Elastic Neural Network

The two-stage transformation process described above, *Compress* and *Grow*, collectively results in a modular, runtime-adaptive architecture that we term an *Elastic Neural Network*. This architecture is not a single fixed model, but rather a collection of *Descendant Networks (DNs)* derived from a common pre-trained backbone, each representing a distinct operating mode along a continuum of resource-accuracy trade-offs.

Each DN corresponds to a specific level of compression and is designed to meet varying deployment constraints. The smallest DN, the *Core Network*, serves as the minimal resource configuration, while successive DNs incorporate more parameters and computational cost via progressive network growth. Importantly, all DNs share a unified parameter structure through the weight-sharing mechanism introduced during the growth stage. This ensures efficient memory utilization and enables seamless switching between modes with minimal overhead. This elastic design allows the *Elastoformer* framework to dynamically adapt its capacity at runtime in response to the system's current state, offering real-time flexibility without the need to store or load multiple independent models.

4.4 System Monitoring and Network Selection

To enable runtime adaptivity, *Elastoformer* includes a lightweight system monitoring and network selection module deployed on the edge device as depicted inside the yellow

block on the bottom of Fig. 3. This module is responsible for selecting the most appropriate Descendant Network (DN) based on current resource availability and use case specific constraints.

During the design phase, each DN is profiled to obtain its corresponding accuracy (%), latency (ms), memory footprint (in MB), and computational cost (in FLOPs). These values are stored as a look-up table on the device. At runtime, the edge system continuously monitors three critical parameters: available memory (M_d), allowable latency (L_d), minimum expected accuracy (A_{min}), and available power budget (P_d). The empirical relationship between power consumption and FLOPs is precomputed and stored as a lookup table to facilitate rapid decision-making. Given the current system state, our framework selects the DN with the lowest computational cost that satisfies all operational constraints as in (10).

$$\begin{aligned} & \min_{\mathcal{DN}} \text{ FLOPs}(\mathcal{DN}) \\ \text{subject to } & \text{Latency}(\mathcal{DN}) \leq L_d \\ & \text{Memory}(\mathcal{DN}) \leq M_d \\ & \text{Accuracy}(\mathcal{DN}) \geq A_{min} \end{aligned} \quad (10)$$

This formulation ensures that the selected network minimizes computation while adhering to the real-time performance and memory limits of the edge device. By combining metadata-aware scheduling with real-time profiling, the *Elastoformer* framework supports seamless and efficient transitions across elastic model configurations.

5 Experimental Setup

5.1 Datasets

The proposed method was evaluated on widely-used benchmark vision datasets: ImageNet [30], CIFAR-10, and CIFAR-100 [19]. The CIFAR-10 dataset consists of 50,000 training images and 10,000 testing images, uniformly categorized into 10 classes, with all images having a resolution of 32×32 . In contrast, the CIFAR-100 dataset contains the same number of training and testing samples as CIFAR-10, but is distributed across 100 fine-grained classes, making it a more challenging classification task. For both CIFAR datasets, standard data augmentation policies such as random cropping, horizontal flipping, and normalization were applied. The ImageNet-1K dataset, comprising 1,281,167 training images and 50,000 validation images spanning 1,000 object categories, was used for large-scale evaluation. In this case, extensive data augmentation strategies—including RandAugment and random erasing—were employed. Regularization techniques such as CutMix [45] and Mixup [46] were also utilized. Additionally, repeated augmentations were applied following the approach of [36].

5.2 Backbone Networks and Baselines

The Vision Transformer Base (ViT-B), comprising 12 layers, an embedding dimension of $d = 768$, and a patch size of 16, as proposed in the original implementation [4], was selected as the pretrained backbone. Comparisons were conducted with several recent approaches for efficient vision transformers [1, 29]. Additionally, comparisons were performed with Early-Exit ViT [40], which provides input-adaptive dynamic inference, and a recent approach [3], in which elasticity is introduced through slimmable feed-forward layers within the transformer blocks.

Further experiments were also conducted on CNNs (ResNet-50 and VGG-16), to evaluate the generalization of the approach beyond transformer-based architectures. The most recent works from each category, reporting results on the same backbone networks, were selected for benchmarking. In particular, the methods presented in [6] were chosen for early-exit networks. Approaches proposed in [9, 39, 44] were selected for their emphasis on adaptivity in dynamic neural networks.

5.3 Evaluation Metrics

The performance of the proposed approach was evaluated in terms of Top-1 accuracy with respect to changes in the total parameters of the descendant network and the variation in FLOPs. Latency of the DNs were also compared. In addition, total memory requirements for storing the networks were assessed, along with the average memory required for switching between different modes of operation. The total GPU hours consumed by the framework when applied to a completely new pre-trained network were also reported. The number of training hours was found to depend on the number of DNs required for a given use case, providing a representative estimate of the overall design time associated with the approach.

5.4 Training Details

Our framework was implemented on PyTorch [28] deep learning framework. All training procedures involved in the experiments were performed using eight NVIDIA GeForce RTX 3090 GPUs, each equipped with 24GB of memory. Each descendant network was trained for 50 epochs, with a linear warmup phase lasting 5 epochs. As previously mentioned, repeated augmentations were utilized, effectively increasing the training epochs by a factor of 3×. An initial learning rate of $3e^{-3}$ was used in conjunction with cosine annealing, unless specified otherwise. The AdamW optimizer was employed for all training processes, with a weight decay of 0.05 applied only to the core network. For all successive network re-training, weight decay was omitted. This was due to the incompatibility of weight decay regularization with

the Partial Weight Freezing mechanism, as it also modified the gradients of shared parameters—an outcome that was undesirable for the proposed framework. The weight update involving weight decay is shown in (11), where w denotes the weight parameter, η represents the learning rate, $\nabla_w \mathcal{L}$ indicates the loss gradient, and λ denotes the weight decay rate. During training, no updates to the shared weights were desired and therefore, the weight decay parameter (λ) was set to zero.

$$w \leftarrow w - \eta \cdot \nabla_w \mathcal{L} - \eta \cdot \lambda w \quad (11)$$

A batch size of 128 was used for both training and evaluation. Stochastic depth [16] was employed to regularize the training process, and selective gradient clipping was applied to prevent exploding gradients and promote faster convergence; however, this was done only on the parameters that were not shared with the previous DNs as discussed in Section 4.2.2.

6 Evaluation and Results

We analyze our approach with focus towards performance and efficiency. We show the performance trade-off with reduced FLOPs, memory footprint and latency, resulting from our unique weight sharing mechanism.

6.1 Performance vs Resource Trade-off

We evaluate the multiple operational modes as shown in Table 1 emerging from the elastic transformation of ViT on Imagenet. We vary the compression ratio (CR) across values in the set $\{0.4, 0.6, 0.8\}$ to yield multiple performance baselines. We observe that our approach enables a flexible trade-off between computational cost (FLOPs) and classification accuracy (Top-1 %). To highlight the generality and adaptability

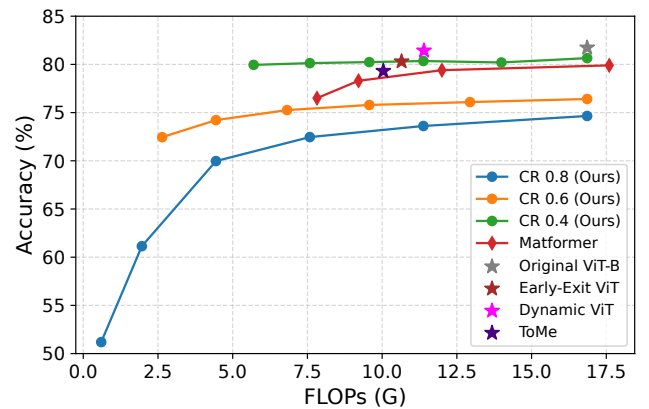


Figure 5: Performance vs. FLOPs trade-off under different compression ratios ($CR \in \{0.4, 0.6, 0.8\}$) on ImageNet

Table 1: Performance vs Resource Trade-off for ViT-B on ImageNet. Six Descendant Networks (DNs) are generated, where Level-1 is the smallest core network and Level-6 matches the original pre-trained network size. Base accuracy = 81.74%, Base FLOPs = 16.86 G.

Compression Ratio	Metric	Level-1	Level-2	Level-3	Level-4	Level-5	Level-6
0.40	ACC % (Top-1)	79.95	80.13	80.24	80.35	80.20	80.13
	FLOPs (G)	5.70	7.58	9.57	11.38	13.99	16.86
0.60	ACC % (Top-1)	72.45	74.22	75.26	75.79	76.09	76.41
	FLOPs (G)	2.64	4.44	6.82	9.57	12.94	16.86
0.80	ACC % (Top-1)	51.19	64.14	69.97	72.46	73.61	74.65
	FLOPs (G)	0.60	1.96	4.44	7.58	11.38	16.86

of our method, we select three representative compression ratios from this set and demonstrate how our approach supports a wide spectrum of accuracy-efficiency trade-offs as observed in Fig. 5, thereby making it well-suited for deployment across devices with varying resource constraints. We observe that our approach performs at par with the original pre-trained network when utilizing the maximum Level-N descendant network, hence providing a fair trade-off. We observe FLOPs reductions by 85% (CR = 0.6) while retaining more than 90% of the peak original performance. With a lower overall compression (CR = 0.4), the smallest core network achieves up to 66% reduction in FLOPs with an accuracy loss $\leq 2\%$, further enhancing low-level (low FLOPs) DN efficiency.

We also compare our approach with an early-exit ViT [40] as shown in Fig. 5, here we need to note even if the performance is equivalent, early-exit networks offers a static memory footprint as the edge device needs to store the full architecture along with some additional exit-layer parameters which presents an additional overhead. We overcome this issue by providing actual memory saving as also discussed in Section 6.3.

Furthermore, we compared with other token merging approaches [1, 29] which provide an efficient ViT with reduced or merged tokens. These methods provide a high performance with a reduced FLOPs requirement, however they fail to provide runtime adaptivity for fluctuating constraints which limits their application for dynamic edge deployments.

We also compare with another recent approach for elastic inference through nested subnetworks [3]. We observe that our approach provides a better tradeoff than Matformer for the similar resource constraint.

We further extended our approach to convolutional neural networks (CNNs), including ResNet-50 [14] and VGG-16 [33], as illustrated in Table 2. We selected, CR = 0.5 for this experiment to generate six descendant networks (DNs),

which offered a balanced trade-off between classification accuracy and computational complexity in terms of FLOPs. Our results reveal that the elastification process preserved the performance of the original pre-trained networks and, in the case of the CIFAR datasets, even led to improved accuracy. We observed a FLOPs reduction up to 75%, representing a substantial efficiency gain that is particularly beneficial for EdgeAI deployment in resource-constrained environments.

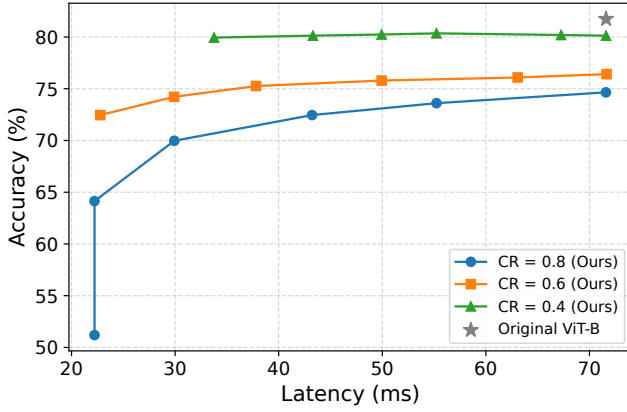
6.2 Performance vs Latency Trade-off

In order to evaluate the effectiveness of the DNs achieved after the elastification, we evaluate the inference latency of the image classification on two platforms with different computation capabilities. We evaluated the latency of generated DNs on Nvidia Jetson Orin which supports an on-device Nvidia Ampere GPU [25] with 8GB memory and on Nvidia Jetson Nano [26] which also comes with an on-device Maxwell GPU having 4GB memory. We estimated the latency for 50 trials with a warmup of 10 trials for the GPU using a Batch Size of 1. We show in Fig. 6 how our elastic transformation can provide a competitive performance under variable latency constraints at the edge server.

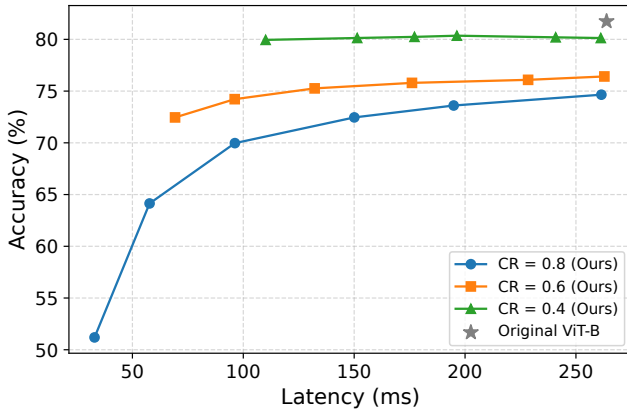
We observe that the latency of the original pre-trained ViT-Base network is more than or equal to the biggest descendant network for all three compression ratio. As shown in Fig. 6, the green curve corresponds to CR = 0.4. In this setting, Elastoformer achieves up to a 50% reduction in latency across both edge platforms, while maintaining performance with less than a 2% drop. We also evaluated the latency for Resnet-50 on Jetson Nano in Fig. 7 to compare with some of the existing works. We observe in Fig. 7 that our approach provides DNs which can work under much stringent latency requirements as compared to other approaches. Our DNs compromise on the accuracy when the FLOPs are on the higher side however other approaches need more FLOPs than the original network itself to maintain higher accuracy.

Table 2: Performance vs Resource Trade-off for ResNet-50 and VGG-16 Descendant Networks (DNs) on benchmark vision datasets. Original network accuracy and FLOPs are indicated.

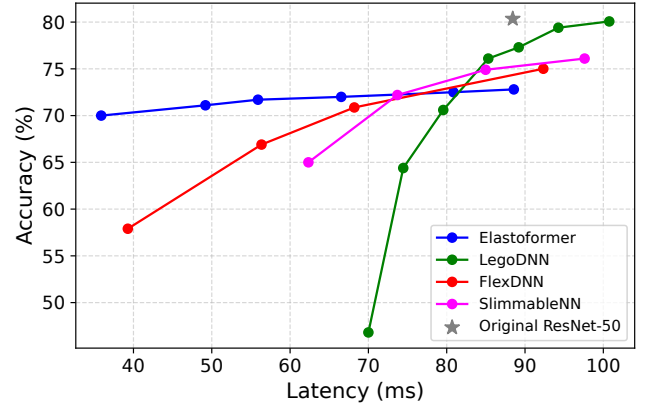
Architecture	Dataset	Metric	DN-1	DN-2	DN-3	DN-4	DN-5	DN-6
ResNet-50 (Orig FLOPs 4.12G)	ImageNet (Orig Acc 80.35%)	ACC (%)	70.00	71.10	71.70	72.00	72.50	72.80
		FLOPs (G)	1.07	1.51	2.03	2.65	3.33	4.12
	CIFAR-10 (Orig Acc 94.37%)	ACC (%)	95.08	94.99	94.91	94.89	94.92	94.85
		FLOPs (G)	1.07	1.51	2.03	2.65	3.33	4.12
	CIFAR-100 (Orig Acc 75%)	ACC (%)	71.16	70.86	70.96	71.29	71.32	70.46
		FLOPs (G)	1.07	1.51	2.03	2.65	3.33	4.12
VGG-16 (Orig FLOPs 15.53G)	CIFAR-10 (Orig Acc 94.16%)	ACC (%)	95.50	95.28	95.29	95.13	95.09	94.86
		FLOPs (G)	3.96	5.60	7.59	9.91	12.51	15.51
	CIFAR-100 (Orig Acc 74%)	ACC (%)	77.78	76.58	77.14	76.62	76.77	76.88
		FLOPs (G)	3.96	5.60	7.59	9.91	12.51	15.51



(a) Nvidia Jetson Orin



(b) Nvidia Jetson Nano

Figure 6: Performance vs. Latency trade-off under different compression ratios ($CR \in 0.4, 0.6, 0.8$) on ImageNet**Figure 7: Performance vs. Latency trade-off of ResNet-50 on Jetson Nano, comparing Elastoformer against other adaptive neural network approaches**

We also estimate the switching overhead latency when transitioning between different DNs on the Jetson Orin, observing a mean switching time of ≈ 50.4 ms. This duration corresponds to approximately one inference cycle, indicating that the effective downtime is limited to a single cycle. As a result, our approach supports continuous inference with negligible impact on sustained throughput, even under frequent runtime adaptations. In contrast, conventional approaches that rely on reloading or retraining models typically incur substantially higher delays, making them impractical for real-time deployment on resource-constrained platforms.

6.3 Memory Overhead Reduction

We have showed in Table 3, the comparison of using our elastic transformation in comparison with AdaptiveNet [39] and the bag of models approach.

Table 3: Memory footprint comparison (including ~1MB metadata per DN).

Base Network (Orig. Size)	Elastoformer (Ours)	AdaptiveNet (SuperNet)	Bag of Models (Accum. DNs)
ViT-B (330.3 MB)	336 MB	—	843.17 MB
ResNet-50 (97.81 MB)	103 MB	381.66 MB	354.72 MB

We observed the memory saving of 76% in comparison with AdaptiveNet for ResNet-50 as per the results reported by them in their original work. In ViT, we notice a memory saving of 60% in comparison to bag of models approach.

Our approach can lead to large reductions in memory as compared to existing approaches with a little overhead of ≈ 1 MB per DN for storing the metadata over the original size of the network. This memory reduction is achieved because of the unique weight sharing mechanism between different DNs, which results in a single set of parameters for multiple modes of operation. This effect would be even more pronounced on larger backbone networks like Large Language Models.

6.4 Elastic Transformation Design Time

We show in Table 4 the total design time required by our approach for elastic transformation of the vision transformer for different number of modes of operation. For training the DNs we have used 8 Nvidia GeForce RTX 3090 GPU's each with a 24GB memory. We note the design time for multiple number of DNs and for two different datasets including Imagenet and CIFAR-100.

Table 4: Design time scaling for different Descendant Networks (DNs)

# DNs	Total Design Time (Hrs)	
	ImageNet	CIFAR-100
4	36	3.5
6	61	5.4
8	85	7.1
11	120	8.4

We observe that the design time scales with both the target dataset and the number of DNs. Empirically, for ImageNet, the design time is $\approx 10N$ hours, where N is the number of DNs. For smaller datasets such as CIFAR-100, the design time is significantly lower, typically ($\leq 10\%$) of that for ImageNet, due to the reduced dataset size.

6.5 Limitations and Future Work

We observe that our core model capacity which is in practice defined through the minimum resources that the edge

system could offer to run the vision application while meeting the latency requirement. We observe that the number of parameters in the core model mostly dictates the performance of the overall system including those of the successive descendant models. We note in Fig 5 that with a higher compression ratio ($CR = 0.8$) the DNs are unable to reach a high performance while with a lower overall compression ($CR = 0.4$), the core model is able to regain back the oracle performance with only 33% of the original FLOPs. We reason that the weight sharing mechanism is working for higher compression ratio which is lucrative for application in Large Language models which have high energy and computation demands and could prove to be a really efficient solution for those architectures.

In future, we aim to develop an automated search algorithm that can determine the optimal number of descendant networks (DNs) and the corresponding compression ratios tailored to specific deployment requirements. Such an approach would enable efficient exploration of the accuracy-efficiency trade-off space with minimal human intervention. Additionally, incorporating a layer-wise distillation loss during the training of DNs could further enhance their performance by transferring knowledge more effectively from the original network to its compressed variants.

7 Conclusion

We presented *Elastoformer*, a novel elastification framework designed to transform static neural architectures, including Vision Transformers (ViTs) into elastic neural networks capable of adapting to dynamic resource constraints in real-time edge environments. The proposed framework generates multiple descendant networks (DNs) from a single pre-trained model, enabling a flexible trade-off between computational cost and predictive performance. Central to our approach is a novel weight sharing mechanism, which reduces memory overhead by up to 76% compared to conventional methods, and lowers inference latency by up to 50% relative to the original ViT. Our empirical evaluations demonstrate that *Elastoformer* achieves a competitive balance between efficiency and accuracy, achieving FLOPs reduction up to 85% while retaining more than 90% of the peak original performance. Furthermore, we show that our framework generalizes effectively to convolutional architectures such as ResNet-50 and VGG-16, underscoring its versatility. The *Elastoformer*

pipeline offers an end-to-end transformation of pre-trained models into elastic networks with minimal retraining, making it a practical and scalable solution for deployment in resource-constrained edge AI systems.

References

- [1] Daniel Bolya, Cheng-Yang Fu, Xiaoliang Dai, Peizhao Zhang, Christoph Feichtenhofer, and Judy Hoffman. 2023. Token Merging: Your ViT but Faster. In *International Conference on Learning Representations*.
- [2] Jiasi Chen and Xukan Ran. 2019. Deep Learning With Edge Computing: A Review. *Proc. IEEE* 107, 8 (August 2019), 1655–1674. doi:10.1109/JPROC.2019.2921977
- [3] Devvrit, Sneha Kudugunta, Aditya Kusupati, Tim Dettmers, Kaifeng Chen, Inderjit Dhillon, Yulia Tsvetkov, Hannaneh Hajishirzi, Sham Kakade, Ali Farhadi, and Prateek Jain. 2025. MatFormer: nested transformer for elastic inference. In *Proceedings of the 38th International Conference on Neural Information Processing Systems* (Vancouver, BC, Canada) (*NIPS '24*). Curran Associates Inc., Red Hook, NY, USA, Article 4461, 30 pages.
- [4] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. 2021. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. *ICLR* (2021).
- [5] Thomas Elsken, Jan Hendrik Metzen, and Frank Hutter. 2019. Neural architecture search: a survey. *J. Mach. Learn. Res.* 20, 1 (Jan. 2019), 1997–2017.
- [6] Biyi Fang, Xiao Zeng, Faen Zhang, Hui Xu, and Mi Zhang. 2020. FlexDNN: Input-Adaptive On-Device Deep Learning for Efficient Mobile Vision. In *2020 IEEE/ACM Symposium on Edge Computing (SEC)*. IEEE, San Jose, CA, USA, 84–95. doi:10.1109/SEC50012.2020.00014
- [7] Biyi Fang, Xiao Zeng, and Mi Zhang. 2018. NestDNN: Resource-Aware Multi-Tenant On-Device Deep Learning for Continuous Mobile Vision. In *Proceedings of the 24th Annual International Conference on Mobile Computing and Networking* (New Delhi, India) (*MobiCom '18*). Association for Computing Machinery, New York, NY, USA, 115–127. doi:10.1145/3241539.3241559
- [8] Gongfan Fang, Xinyin Ma, Mingli Song, Michael Bi Mi, and Xinchao Wang. 2023. Depgraph: Towards any structural pruning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 16091–16101.
- [9] Rui Han, Qinglong Zhang, Chi Harold Liu, Guoren Wang, Jian Tang, and Lydia Y. Chen. 2021. LegoDNN: block-grained scaling of deep neural networks for mobile vision. In *Proceedings of the 27th Annual International Conference on Mobile Computing and Networking* (New Orleans, Louisiana) (*MobiCom '21*). Association for Computing Machinery, New York, NY, USA, 406–419. doi:10.1145/3447993.3483249
- [10] Song Han, Huizi Mao, and William J. Dally. 2016. Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding. doi:10.48550/arXiv.1510.00149 arXiv:1510.00149 [cs].
- [11] Song Han, Jeff Pool, John Tran, and William J. Dally. 2015. Learning both weights and connections for efficient neural networks. In *Proceedings of the 29th International Conference on Neural Information Processing Systems - Volume 1* (Montreal, Canada) (*NIPS'15*). MIT Press, Cambridge, MA, USA, 1135–1143.
- [12] Yizeng Han, Gao Huang, Shiji Song, Le Yang, Honghui Wang, and Yulin Wang. 2022. Dynamic Neural Networks: A Survey. *IEEE Transactions on Pattern Analysis & Machine Intelligence* 44, 11 (Nov. 2022), 7436–7456. doi:10.1109/TPAMI.2021.3117837
- [13] B. Hassibi, D.G. Stork, and G.J. Wolff. 1993. Optimal Brain Surgeon and general network pruning. In *IEEE International Conference on Neural Networks*. 293–299 vol.1. doi:10.1109/ICNN.1993.298572
- [14] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep Residual Learning for Image Recognition. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 770–778. doi:10.1109/CVPR.2016.90
- [15] Andrew G. Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. 2017. MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications. doi:10.48550/arXiv.1704.04861 arXiv:1704.04861 [cs].
- [16] Gao Huang, Yu Sun, Zhuang Liu, Daniel Sedra, and Kilian Q. Weinberger. 2016. Deep Networks with Stochastic Depth. In *Computer Vision – ECCV 2016*, Bastian Leibe, Jiri Matas, Nicu Sebe, and Max Welling (Eds.). Springer International Publishing, Cham, 646–661.
- [17] Itay Hubara, Matthieu Courbariaux, Daniel Soudry, Ran El-Yaniv, and Yoshua Bengio. 2016. Binarized neural networks. In *Proceedings of the 30th International Conference on Neural Information Processing Systems* (Barcelona, Spain) (*NIPS'16*). Curran Associates Inc., Red Hook, NY, USA, 4114–4122.
- [18] Junchen Jiang, Ganesh Ananthanarayanan, Peter Bodik, Siddhartha Sen, and Ion Stoica. 2018. Chameleon: scalable adaptation of video analytics. In *Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication*. ACM, Budapest Hungary, 253–266. doi:10.1145/3230543.3230574
- [19] Alex Krizhevsky. 2009. *Learning Multiple Layers of Features from Tiny Images*. Technical Report. University of Toronto. <https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf> Technical Report.
- [20] Stefanos Laskaridis, Alexandros Kouris, and Nicholas D. Lane. 2021. Adaptive Inference through Early-Exit Networks: Design, Challenges and Directions. In *Proceedings of the 5th International Workshop on Embedded and Mobile Deep Learning*. ACM, Virtual WI USA, 1–6. doi:10.1145/3469116.3470012
- [21] Yann Le Cun, John S. Denker, and Sara A. Solla. 1989. Optimal brain damage. In *Proceedings of the 3rd International Conference on Neural Information Processing Systems* (*NIPS'89*). MIT Press, Cambridge, MA, USA, 598–605.
- [22] Hao Li, Asim Kadav, Igor Durdanovic, Hanan Samet, and Hans Peter Graf. 2017. Pruning Filters for Efficient ConvNets. doi:10.48550/arXiv.1608.08710 arXiv:1608.08710 [cs].
- [23] Svetlana Minakova, Dolly Sapra, Todor Stefanov, and Andy D. Pimentel. 2022. Scenario Based Run-Time Switching for Adaptive CNN-Based Applications at the Edge. *ACM Transactions on Embedded Computing Systems* 21, 2 (March 2022), 1–33. doi:10.1145/3488718
- [24] Pavlo Molchanov, Stephen Tyree, Tero Karras, Timo Aila, and Jan Kautz. 2017. Pruning Convolutional Neural Networks for Resource Efficient Inference. doi:10.48550/arXiv.1611.06440 arXiv:1611.06440 [cs].
- [25] NVIDIA. 2025. Jetson Orin Embedded Systems. <https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/jetson-orin/> Accessed: Jun. 16, 2025.
- [26] NVIDIA Corporation. 2025. Jetson Nano: Product Development. <https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/jetson-nano/product-development/>. Accessed: 2025-06-27.
- [27] Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. 2013. On the difficulty of training recurrent neural networks. In *Proceedings of the 30th International Conference on Neural Information Processing Systems - Volume 28* (Atlanta, GA, USA) (*ICML'13*). JMLR.org, III–1310–III–1318.

- [28] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. 2019. *PyTorch: an imperative style, high-performance deep learning library*. Curran Associates Inc., Red Hook, NY, USA.
- [29] Yongming Rao, Wenliang Zhao, Benlin Liu, Jiwen Lu, Jie Zhou, and Cho-Jui Hsieh. 2021. DynamicViT: efficient vision transformers with dynamic token sparsification. In *Proceedings of the 35th International Conference on Neural Information Processing Systems (NIPS '21)*. Curran Associates Inc., Red Hook, NY, USA, Article 1068, 13 pages.
- [30] Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, Michael Bernstein, Alexander C. Berg, and Li Fei-Fei. 2015. ImageNet Large Scale Visual Recognition Challenge. *Int. J. Comput. Vision* 115, 3 (Dec. 2015), 211–252. doi:10.1007/s11263-015-0816-y
- [31] Dolly Sapra and Andy D. Pimentel. 2020. Constrained Evolutionary Piecemeal Training to Design Convolutional Neural Networks. In *Trends in Artificial Intelligence Theory and Applications. Artificial Intelligence Practices: 33rd International Conference on Industrial, Engineering and Other Applications of Applied Intelligent Systems, IEA/AIE 2020, Kitakyushu, Japan, September 22–25, 2020, Proceedings* (Kitakyushu, Japan). Springer-Verlag, Berlin, Heidelberg, 709–721. doi:10.1007/978-3-030-55789-8_61
- [32] Vasuki Shankar. 2024. Edge AI: A Comprehensive Survey of Technologies, Applications, and Challenges. In *2024 1st International Conference on Advanced Computing and Emerging Technologies (ACET)*. doi:10.1109/ACET61898.2024.10730112
- [33] Karen Simonyan and Andrew Zisserman. 2015. Very Deep Convolutional Networks for Large-Scale Image Recognition. doi:10.48550/arXiv.1409.1556 arXiv:1409.1556 [cs].
- [34] Wenhao Sun, Grace Li Zhang, Xunzhao Yin, Cheng Zhuo, Huaxi Gu, Bing Lil, and Ulf Schlichtmann. 2023. SteppingNet: A Stepping Neural Network with Incremental Accuracy Enhancement. In *2023 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, Antwerp, Belgium, 1–6. doi:10.23919/DATE56975.2023.10136943
- [35] Surat Teerapittayanon, Bradley McDanel, and H.T. Kung. 2016. BranchyNet: Fast inference via early exiting from deep neural networks. In *2016 23rd International Conference on Pattern Recognition (ICPR)*. 2464–2469. doi:10.1109/ICPR.2016.7900006
- [36] Hugo Touvron, Matthieu Cord, Matthijs Douze, Francisco Massa, Alexandre Sablayrolles, and Herve Jegou. 2021. Training data-efficient image transformers amp; distillation through attention. In *Proceedings of the 38th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 139)*, Marina Meila and Tong Zhang (Eds.). PMLR, 10347–10357. <https://proceedings.mlr.press/v139/touvron21a.html>
- [37] Huan Wang, Can Qin, Yue Bai, and Yun Fu. 2023. Why is the State of Neural Network Pruning so Confusing? On the Fairness, Comparison Setup, and Trainability in Network Pruning. doi:10.48550/arXiv.2301.05219 arXiv:2301.05219 [cs].
- [38] Yulin Wang, Rui Huang, Shiji Song, Zeyi Huang, and Gao Huang. 2021. Not all images are worth 16x16 words: dynamic transformers for efficient image recognition. In *Proceedings of the 35th International Conference on Neural Information Processing Systems (NIPS '21)*. Curran Associates Inc., Red Hook, NY, USA, Article 915, 14 pages.
- [39] Hao Wen, Yuanchun Li, Zunshuai Zhang, Shiqi Jiang, Xiaozhou Ye, Ye Ouyang, Yaqin Zhang, and Yunxin Liu. 2023. AdaptiveNet: Post-deployment Neural Architecture Adaptation for Diverse Edge Environments. In *Proceedings of the 29th Annual International Conference on Mobile Computing and Networking (Madrid, Spain) (ACM MobiCom '23)*. Association for Computing Machinery, New York, NY, USA, Article 28, 17 pages. doi:10.1145/3570361.3592529
- [40] Guanyu Xu, Jiawei Hao, Li Shen, Han Hu, Yong Luo, Hui Lin, and Jialie Shen. 2023. LGViT: Dynamic Early Exiting for Accelerating Vision Transformer. In *Proceedings of the 31st ACM International Conference on Multimedia (Ottawa ON, Canada) (MM '23)*. Association for Computing Machinery, New York, NY, USA, 9103–9114. doi:10.1145/3581783.3611762
- [41] Yifan Xu, Zhijie Zhang, Mengdan Zhang, Kekai Sheng, Ke Li, Weiming Dong, Liqing Zhang, Changsheng Xu, and Xing Sun. 2022. Evo-vit: Slow-fast token evolution for dynamic vision transformer. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 36. 2964–2972.
- [42] Huanrui Yang, Hongxu Yin, Maying Shen, Pavlo Molchanov, Hai Li, and Jan Kautz. 2023. Global Vision Transformer Pruning with Hessian-Aware Saliency. In *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 18547–18557. doi:10.1109/CVPR52729.2023.01779
- [43] Jiahui Yu and Thomas Huang. 2019. Universally Slimmable Networks and Improved Training Techniques. In *2019 IEEE/CVF International Conference on Computer Vision (ICCV)*. 1803–1811. doi:10.1109/ICCV.2019.00189
- [44] Jiahui Yu, Linjie Yang, Ning Xu, Jianchao Yang, and Thomas Huang. 2018. Slimmable Neural Networks. doi:10.48550/arXiv.1812.08928 arXiv:1812.08928 [cs].
- [45] Sangdoo Yun, Dongyoon Han, Sanghyuk Chun, Seong Joon Oh, Youngjoon Yoo, and Junsuk Choe. 2019. CutMix: Regularization Strategy to Train Strong Classifiers With Localizable Features. In *2019 IEEE/CVF International Conference on Computer Vision (ICCV)*. 6022–6031. doi:10.1109/ICCV.2019.00612
- [46] Hongyi Zhang, Moustapha Cisse, Yann N. Dauphin, and David Lopez-Paz. 2018. mixup: Beyond Empirical Risk Minimization. doi:10.48550/arXiv.1710.09412 arXiv:1710.09412 [cs].
- [47] Xiangyu Zhang, Xinyu Zhou, Mengxiao Lin, and Jian Sun. 2018. ShuffleNet: An Extremely Efficient Convolutional Neural Network for Mobile Devices. In *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE Computer Society, Los Alamitos, CA, USA, 6848–6856. doi:10.1109/CVPR.2018.00716
- [48] Mingjian Zhu, Yehui Tang, and Kai Han. 2021. Vision Transformer Pruning. doi:10.48550/arXiv.2104.08500 arXiv:2104.08500 [cs].

Received 20 June 2025; accepted 22 September 2025